

PROJET WEB – CAHIER DES CHARGES

Application de Gestion de Notes Personnelles

Champ	Détail
Durée de réalisation	1 semaine (≈ 20h de travail effectif)
Mode de travail	Individuel ou en binôme
Technologies principales	Laravel 10/11 + React 18
Base de données	SQLite ou MySQL
Authentification	Laravel Sanctum (tokens)
Style imposé	Tailwind, Bootstrap ou CSS – Responsive
Rendu	Code source + Compte rendu PDF (4–6 pages)

1. Contexte et Objectif

Ce projet consiste à développer une application web complète permettant à un utilisateur de s'authentifier et de gérer ses notes personnelles (type « pense-bête »).

Chaque note contient les informations suivantes :

- Un titre (obligatoire, max 100 caractères)
- Un contenu textuel (optionnel, texte long)
- Une date de création (générée automatiquement)
- Un niveau de priorité : Basse / Moyenne / Haute

2. Objectifs Pédagogiques

À l'issue du projet, l'étudiant sera capable de :

- Mettre en œuvre une API REST avec Laravel (routes, contrôleurs, Eloquent, validation)
- Réaliser une interface utilisateur dynamique avec React (composants fonctionnels, hooks, état)
- Gérer l'authentification par token via Laravel Sanctum ou JWT
- Communiquer entre frontend et backend avec Axios (requêtes AJAX)
- Produire un compte rendu structuré (analyse, conception, bilan)

3. Spécifications Fonctionnelles

3.1 Authentification

- Page d'inscription : email, mot de passe, nom d'utilisateur
- Page de connexion : email + mot de passe
- Accès aux notes réservé aux utilisateurs authentifiés
- Bouton de déconnexion : détruit le token et redirige vers /login

3.2 Gestion des Notes (CRUD complet)

Opération	Description
Lister (Read)	Afficher toutes les notes de l'utilisateur connecté, triées par date de création décroissante
Ajouter (Create)	Formulaire : titre (requis, max 100 car.), contenu (optionnel), priorité (select)
Modifier (Update)	Pré-remplir le formulaire avec les données existantes de la note
Supprimer (Delete)	Demande de confirmation avant la suppression définitive

Chaque note affiche :

- Sa date de création dans un format lisible (ex : 3 mai 2025)
- Un badge de couleur selon la priorité (vert / orange / rouge)

3.3 Validation et Retours Utilisateur

- Messages de succès ou d'erreur (toasts ou alertes stylisées) après chaque action
- Blocage de la soumission si le titre est vide
- Gestion des erreurs API : 401 (non autorisé), 404 (non trouvé), 422 (validation)

4. Contraintes Techniques

Côté	Technologies / Règles imposées
Backend	Laravel 10 ou 11 – API REST – Sanctum ou JWT – SQLite ou MySQL
Frontend	React 18 (CRA ou Vite) – React Router – Axios – Hooks (useState, useEffect, useContext)
Style	Tailwind CSS, Bootstrap ou CSS simple – Interface responsive obligatoire
Communication	Toutes les opérations via requêtes AJAX (aucun rechargement de page)

5. Structure Attendue de l'Application

5.1 Backend Laravel

Modèles Eloquent

- User (fourni par Laravel)
- Note – champs : id, title, content, priority, user_id, created_at, updated_at

Migrations & Seeders

- Migration pour la table notes
- Seeder : 1 utilisateur de test + quelques notes d'exemple

Routes API

Méthode	Endpoint	Action	Middleware
POST	/api/register	Inscription de l'utilisateur	—
POST	/api/login	Connexion (retourne le token)	—
POST	/api/logout	Déconnexion (révoque le token)	auth:sanctum
GET	/api/notes	Liste des notes de l'utilisateur	auth:sanctum
POST	/api/notes	Création d'une note	auth:sanctum
PUT	/api/notes/{id}	Modification d'une note	auth:sanctum
DELETE	/api/notes/{id}	Suppression d'une note	auth:sanctum

5.2 Frontend React

Pages

- /login – Page de connexion (publique)
- /register – Page d'inscription (publique)
- /notes – Liste des notes + formulaire (privée, protégée)

Composants

Composant	Rôle
NoteList	Affiche la liste complète des notes de l'utilisateur
NoteForm	Formulaire d'ajout ou d'édition d'une note (avec Axios)
NoteItem	Représentation d'une note avec boutons Modifier / Supprimer

Gestion de l'état

- Option A : useState + useEffect avec rechargement manuel après chaque action
- Option B : useContext pour partager le token et la fonction fetchNotes à travers les composants

6. Contenu du Compte Rendu (PDF, 4–6 pages)

Section	Contenu attendu
1. Introduction	Objectif du projet, contexte, aperçu des fonctionnalités
2. Architecture technique	Schéma client React ↔ API Laravel ↔ BDD + justification des choix
3. Captures commentées	Inscription/connexion, liste des notes, formulaire ajout/édition, confirmation suppression
4. Extraits de code	NoteController@store (Laravel), NoteForm avec Axios (React), config Sanctum
5. Difficultés & solutions	Gestion du token, CORS, mise à jour de la liste après suppression, etc.
6. Tests manuels	Tableau : action attendue / résultat obtenu
7. Bilan & évolutions	Apprentissages + 2-3 pistes d'amélioration (tags, recherche, partage)

7. Critères d'Évaluation

Critère	Points
Fonctionnalités complètes (CRUD + authentification)	5
Respect des contraintes techniques (API REST, React sans reload)	3
Qualité du code (structure, nommage, lisibilité)	2
Interface utilisateur (clarté, responsive, retours visuels)	2
Compte rendu (clarté, illustrations, analyse des difficultés)	4
Rendu dans les délais	1
TOTAL	17
Bonus (ex : filtre par priorité)	+ 3 pts max

Note : Le bonus de 3 points est accordé pour toute fonctionnalité supplémentaire cohérente, par exemple : filtre par priorité, recherche de notes, ou export.

8. Livrables Attendus

- Code source complet (backend Laravel + frontend React), idéalement sur un dépôt Git (GitHub/GitLab)
- Fichier .env.example configuré (sans informations sensibles)
- Compte rendu au format PDF (4 à 6 pages)
- Instructions de lancement (README.md) : étapes d'installation, commandes npm / artisan

Bonne réalisation à toutes et à tous !